8.7 中断的概念

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.7）｜重要度 ★★★★｜常见考法：选择题（内中断/外中断、可屏蔽/不可屏蔽、向量/非向量、单级/多重）

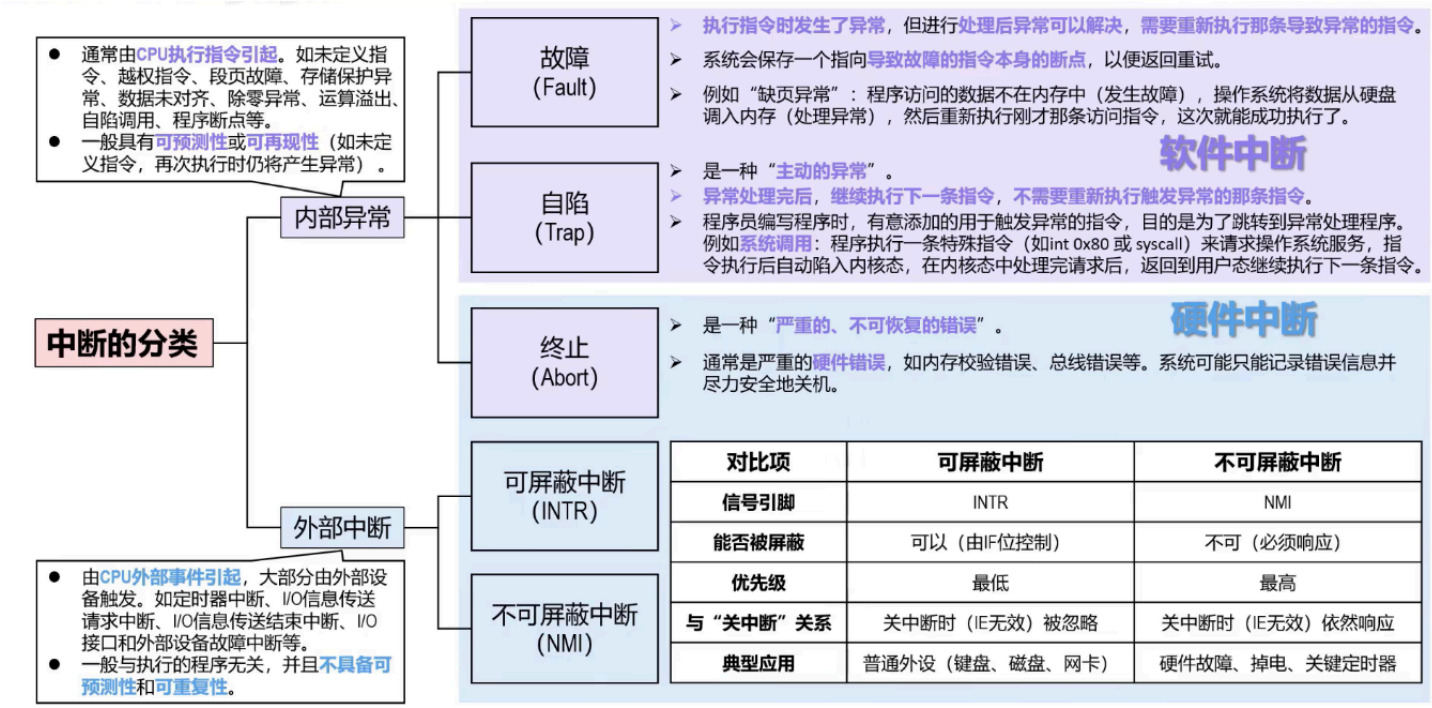
一句话核心：中断是 CPU 暂停当前程序、转去处理更紧急事件的机制；按来源分内/外，按识别分向量/非向量，按嵌套分单级/多重

一、中断的作用



作用	说明
提升并行性	外部设备主动“敲门”通知 CPU，使操作系统能在设备准备阶段调度 CPU 执行其他任务，实现 CPU 与外部设备的并行工作
支持程序调试	通过设置断点、单步执行暂停程序，便于开发者观察中间结果和状态
实现故障与异常处理	硬件故障或软件异常触发中断，CPU 立即调用相应处理程序，提高系统可靠性
保障实时处理	适用于现场数据采集、机器人控制等实时场景，利用中断的“打断”特性确保关键数据及时处理
方便人机交互	键盘、鼠标等外部输入均通过中断方式通知系统
实现多任务调度	中断技术是多任务操作系统的基础机制之一；操作系统利用定时中断强制收回 CPU 控制权，实现时间片轮转
支持多处理机系统交互	不同处理器之间通过中断方式实现信息交换和任务切换

二、中断的分类



2.1 内中断（异常）

由 CPU 执行指令内部产生，与当前指令同步，一律不可屏蔽

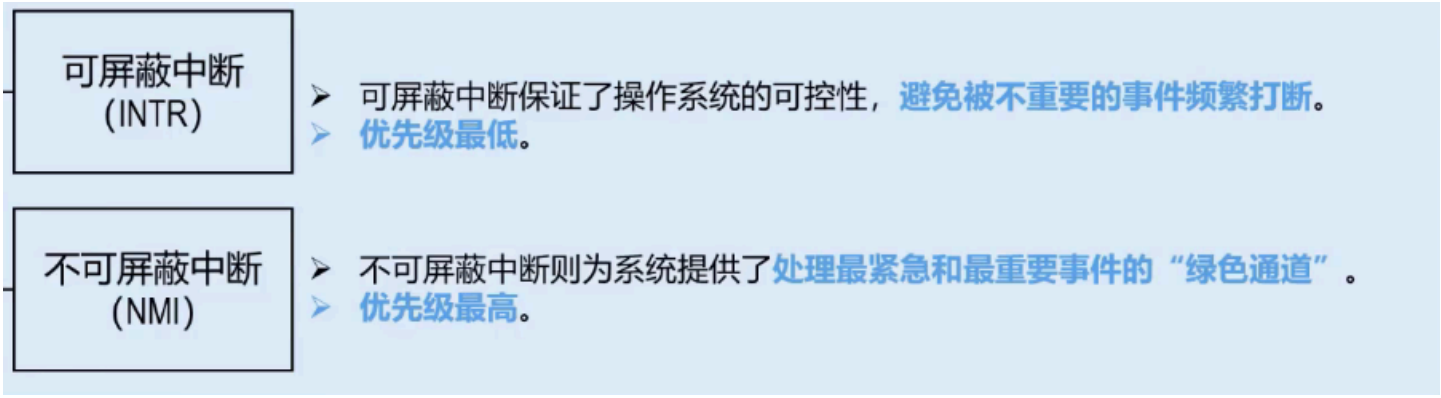
类型	含义	典型例子
故障（Fault）	指令执行过程中发生异常，但可能可以恢复，需要重新执行该指令	缺页、非法指令、除数为 0、溢出、越界、数据未对齐
自陷（Trap）	一种“软件中断”，由程序主动触发，用于系统调用、断点调试、单步执行	系统调用、断点、单步
终止（Abort）	严重的、不可恢复的故障，终止当前程序	硬件故障、控制器出错、总线错误

- 故障：处理后返回并重新执行引发故障的指令
- 自陷：处理后通常返回到下一条指令
- 终止：无法恢复，只能终止程序
- 408 常把“除 0、缺页、非法指令、溢出、越界、单步”混在一起考分类

2.2 外中断（中断）

由 CPU 外部设备发出，与当前指令异步，在指令执行完毕的边界检测

类型	触发方式	是否可屏蔽	优先级	典型例子
可屏蔽中断（INTR）	外部设备通过 INTR 线向 CPU 发请求	受 IF 位控制，IF=0 时被屏蔽	最低	键盘、打印机、定时器
不可屏蔽中断（NMI）	外部紧急事件通过 NMI 线向 CPU 发请求	不受 IF 位控制	最高	掉电、硬件故障、严重错误



- 可屏蔽中断保证操作系统的可控性，避免被不重要的事件频繁打断
- 不可屏蔽中断为系统提供处理最紧急、最重要事件的“绿色通道”
- IF 只对 INTR 有效；NMI 和内中断不受 IF 影响

2.3 内中断 vs 外中断

分类维度	内中断（异常）	外中断（中断）
触发来源	CPU 执行指令内部产生	CPU 外部设备发出
是否与指令相关	同步，与当前指令绑定	异步，与指令无关
是否可屏蔽	一律不可屏蔽	INTR 可屏蔽，NMI 不可屏蔽
典型例子	除 0、非法指令、溢出、缺页、单步、越界、数据未对齐	键盘/打印机等 I/O 中断、时钟中断、掉电
检测时机	指令执行过程中检测	每条指令执行完毕的边界检测

三、向量中断和非向量中断

向量中断和非向量中断

向量中断

- 每种中断源预先分配唯一的**向量地址**，存储在**中断向量表**或**硬件寄存器**中。
- 设备**触发中断后**，硬件自动根据中断源识别对应的向量地址。
- CPU直接加载向量地址指向的**中断服务程序（ISR）**，无需额外查询步骤，响应速度快。
- 例如，x86系统中，外部中断通过可编程中断控制器（PIC）生成向量号，CPU据此从内存中读取ISR地址并执行。

非向量中断

- 多个设备**可能共享同一中断请求线**，硬件仅能识别“有中断发生”，无法直接定位具体设备。
- CPU响应中断后，需通过**软件**逐个查询设备状态寄存器，确定触发中断的设备。
- 根据查询结果，通过**条件分支指令**跳转到对应设备的ISR，流程相对繁琐，响应速度较慢。
- 例如，早期ARM处理器中，外部中断请求（IRQ）可能由多个外设共享，需在中断服务程序（ISR）开头通过读取外设状态寄存器判断具体中断源。

维度	向量中断	非向量中断
识别方式	每个中断源预先分配唯一的向量地址，硬件直接给出	多个设备共享同一中断请求线，硬件只能识别“有中断发生”
定位过程	CPU 直接加载向量地址指向的 ISR，无需额外查询	CPU 响应后，软件逐个查询设备状态寄存器，确定中断源
响应速度	快	慢，流程繁琐
典型场景	x86 系统中，外部中断通过可编程中断控制器（PIC）生成向量号	早期 ARM 处理器中，多个外设共享 IRQ，需在 ISR 开头读取状态寄存器判断具体中断源

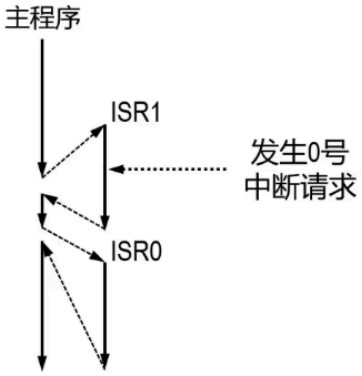
- 向量中断的关键：硬件知道“是谁”，直接给入口
- 非向量中断的关键：硬件只知道“有人来了”，软件自己查“是谁”

四、单级中断和多重中断

单级中断和多重中断（嵌套中断）

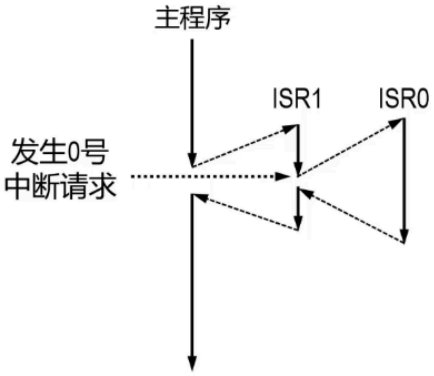
- 对于**单级中断**，执行中断服务程序（ISR）时，不再响应其他中断请求。
- 对于**多重中断**，执行中断服务程序（ISR）时，可以被更高优先级的中断请求打断，也称为**嵌套中断**。

单级中断和**多重中断**都可以有多个中断源和中断优先级，二者的区别在于多个中断之间能否嵌套。



单级中断示意图

先被CPU响应的ISR先完成



多重中断（嵌套中断）示意图

先被CPU响应的ISR不一定先完成

维度	单级中断	多重中断（嵌套中断）
是否允许嵌套	执行 ISR 时不再响应其他中断	执行 ISR 时可以被更高优先级的中断请求打断
开中断位置	ISR 全程关中断，末尾才开	保护现场后开中断，允许嵌套；恢复现场前关中断
完成顺序	先被 CPU 响应的 ISR 先完成	先被 CPU 响应的 ISR 不一定先完成
硬件代价	小	需要优先级判别逻辑（如 8259A 的 PR）

- 单级中断和多重中断都可以有多个中断源和中断优先级
- 二者的区别只有一个：多个中断之间能否嵌套
- 单级中断中，先响应的先完成；多重中断中，高优先级可以“后来居上”

五、易错点

- 内中断不等于“CPU 内部硬件故障”：内中断包括故障、自陷、终止，自陷是软件主动触发的
- 外中断不等于“所有外部事件”：外部设备的中断是外中断，但缺页、除 0 等是内中断
- 可屏蔽中断与不可屏蔽中断的区别是“是否受 IF 控制”，不是“是否重要”
- 向量中断与非向量中断的区别是“谁负责识别中断源”，不是“有没有中断向量表”
- 单级中断不是“只有一个中断源”，而是“ISR 执行期间不响应新中断”
- 多重中断不是“多个中断同时发生”，而是“ISR 执行期间允许更高优先级中断嵌套”
- 中断响应发生在指令执行完毕的边界，不会在指令执行中途插入可屏蔽中断

六、考法透视

选择题常见设错点

1. 把缺页、除 0、系统调用归为外中断
2. 认为 NMI 也受 IF 位控制
3. 认为可屏蔽中断一定比不可屏蔽中断重要
4. 认为单级中断只能有一个中断源
5. 认为多重中断中先响应的 ISR 一定先完成
6. 把“向量地址”当成“中断服务程序入口地址”
7. 认为中断可以在指令执行中途响应

辨析题

- 给一个事件，判断内/外中断、可屏蔽/不可屏蔽、故障/自陷/终止
- 给一个中断系统，判断向量/非向量、单级/多重
- 比较中断与异常、中断与子函数调用

七、知识关联

- 前置：8.6 中断驱动方式概述（为什么需要中断）
- 后续：8.8 中断优先级和中断屏蔽（谁能打断谁）、8.9 中断请求（中断源如何被识别）、8.10 单级中断处理流程、8.11 多重中断的处理流程
- OS 视角：中断与系统调用、进程调度、时钟中断
- 全章导航：8.0 导航与术语

八、费曼自测

1. 中断按来源分哪几类 → 内中断（异常）：故障、自陷、终止；外中断：可屏蔽 INTR、不可屏蔽 NMI
2. 可屏蔽中断和不可屏蔽中断的核心区别是什么 → 是否受 IF 位控制；INTR 受 IF 控制，NMI 不受 IF 控制且优先级最高
3. 向量中断和非向量中断的区别是什么 → 向量中断由硬件直接给出向量地址，快速定位 ISR；非向量中断由软件查询状态寄存器确定中断源
4. 单级中断和多重中断的区别是什么 → 单级中断执行 ISR 时不再响应其他中断；多重中断允许更高优先级中断打断当前 ISR
5. 中断响应发生在什么时机 → 一条指令执行完毕的边界；可屏蔽中断不会在指令执行中途插入